Counter-increment wiring — `helix_proxy_acl_decisions_total` + `helix_proxy_tunnel_down_responses_total`

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Design (task #56 hardening — byte-path→api counter increment) **Authority:** consumer design doc under §11.4.45 / §11.4.108 (wiring-before-behaviour). Constitution wins on any conflict. **Scope:** how the two pre-registered-at-0 control-API counters become live on real proxied traffic, consistent with the committed `helix_proxy_vpn_up`-from-Redis pattern.

---

1. Problem statement (FACTS)

The control-API pre-registers two counters at `0` and honestly defers their increment:

- `control-plane/internal/api/metrics.go:14-17` — *"The two counters are registered + exposed here even though they are incremented by the acl-helper / byte-path later (plan P5/P10)… the increment seam (`Metrics.Inc*`) is ready."*
- `control-plane/internal/api/metrics.go:74-82` — `IncACLDecision(decision)` / `IncTunnelDownResponse()` are in-process `CounterVec`/`Counter` mutators.
- The §56 live guard `tests/observability/metrics_scrape_test.sh:225-265` already drives ONE real proxied request (`curl -x $PROXY_URL $PROBE_TARGET`, line 229-230), re-scrapes `/metrics`, and computes the `acl_decisions_total` delta. Today the delta is `0` → an honest `feature_disabled_by_config` SKIP (line 260-265). With `HELIX_METRICS_BYTEPATH_WIRED=1` a flat counter becomes a **HARD FAIL** (line 250-254).

The gap: the two counters never increment on real traffic, so the §56 test can only SKIP the increment sub-proof.

2. Architecture constraint — two processes, one shared store (FACTS)

Fact	Evidence
The acl-helper is a SEPARATE, Squid-spawned `external_acl_type` helper process.	`control-plane/cmd/acl-helper/main.go:1-8`; `control-plane/internal/routing/routing.go:281` (`external_acl_type vpn_route … {{.HelperPath}}`).
The acl-helper already holds a Redis client and nothing else.	`cmd/acl-helper/main.go:13` (*"Dependencies: stdlib + the committed internal/redis only"*), `main.go:70` (`redis.Open`).
The control-API is a LONG-LIVED, separate process holding the Prometheus registry in memory.	`internal/api/server.go:26-49` (`server.metrics = newMetrics(...)`, one registry per server).
The control-API is ALSO already Redis-connected in every deployment.	`docker-compose.observability.yml:109` (`REDIS_ADDR=proxy-redis:6379`), passed to `NewServer(... bus redis.StatusBus ...)` `server.go:44-46`.
The two processes share **only Redis** — no shared memory, no shared registry.	The acl-helper is baked into the Squid image (`docker-compose.dynamic.yml:223 --helper-path …`); the API runs as `proxy-api`.
The API already sources a live metric fail-closed FROM Redis at scrape time.	`internal/api/metrics.go:89-123` `vpnUpCollector.Collect` reads `vpn:status:<profile>` per scrape.

Consequence: `Metrics.IncACLDecision` (an in-process call inside the API) can *never* be driven by the acl-helper, which lives in a different process and never touches the API's registry. The only wiring surface between "sees the traffic" (acl-helper) and "owns the metric" (API) is **Redis**.

3. The OK / ERR / tunnel_down mapping (FACT — 1:1)

The Squid dynamic include binds the helper verdict to the response:

```
external_acl_type vpn_route … {{.HelperPath}}      #
routing.go:281
acl tun_up external vpn_route                       #
routing.go:282
http_access deny !tun_up                            #
routing.go:295  (helper ERR ⇒ denied)
deny_info 503:ERR_TUNNEL_DOWN tun_up                #
routing.go:297  (that deny ⇒ 503 page)
```

The acl-helper's pure verdict (`internal/aclhelper/decide.go:47-70`) returns `OK tag=<tunnel>` on the single affirmative path (route exists AND tunnel `StateUp`); **every** other case returns `ERR`. Because `deny_info 503:ERR_TUNNEL_DOWN` is bound **only** to `tun_up`, every helper `ERR` → exactly one `ERR_TUNNEL_DOWN` 503 response, and no other Squid 5xx maps to that page. Therefore:

- `helix_proxy_acl_decisions_total{decision="OK"}` = count of helper OK verdicts.
- `helix_proxy_acl_decisions_total{decision="ERR"}` = count of helper ERR verdicts.
- `helix_proxy_tunnel_down_responses_total` = count of helper ERR verdicts (1:1 with the served 503).

Honest boundary (§11.4.6): the count is of *helper ERR emissions*, which the committed `deny_info` config maps 1:1 to served `ERR_TUNNEL_DOWN` 503s. A hypothetical Squid-internal 5xx that is NOT an `ERR_TUNNEL_DOWN` page is correctly NOT counted (it is a different error). This is a faithful proxy for "503s served", not a guess.

4. Options (ranked)

Option A1 — acl-helper INCRs a Redis counter; API reads it at scrape (RECOMMENDED)

The acl-helper `INCR`s a per-decision Redis counter key after it answers Squid. The API replaces the in-process counters with a **Redis-reading const-metric collector** that mirrors `vpnUpCollector` exactly, emitting `prometheus.CounterValue` const metrics read from those keys at scrape time.

- **Mirrors the confirmed pattern** (`vpnUpCollector`, `metrics.go:89-123`) — one architecture, not two.
- **Keeps the acl-helper's dependency surface** (Redis only, `main.go:13`) — no new import, no new ingress.
- **Lossless** when Redis is up — `INCR` is atomic and durable, unlike fire-and-forget pub/sub.
- **Survives API restart** — the count lives in Redis, so an API restart does NOT spuriously reset the Prometheus counter (an

in-process `CounterVec` would). Redis being flushed is the only reset, which is normal.
- **Multi-replica-consistent** — every API replica reads the same key and scrapes the same value.

Option A2 — acl-helper `PublishEvent` on a Redis channel; API subscribes and calls `Inc*`

Reuses `PublishEvent`/`SubscribeEvents` (`internal/redis/client.go:167-215`). Keeps counters in-process (native monotonic) but adds a background subscriber goroutine to the API. **Rejected as primary:** pub/sub is lossy — a decision published while the API is down/reconnecting is dropped → silent undercount (a §11.4-relevant inaccuracy for a counter). More moving parts than A1 for a worse guarantee.

Option B — acl-helper POSTs to a new API HTTP endpoint per decision

Rejected. (1) Adds a synchronous network call + new failure mode into the decision-adjacent path of a process whose whole point is "out of the byte path, fails closed" (`main.go:6-8`). (2) Couples the acl-helper to the API's location + the mTLS control port (the plaintext `/metrics` listener is GET-only, `server.go:110-114`). (3) Violates the stated acl-helper dependency constraint (stdlib + internal/redis only, `main.go:13`) and adds a new ingress surface to the API.

5. Concrete code change (Option A1)

5.1 `internal/redis/redis.go` — key scheme + contract (resolve-by-name §11.4.111)

Add beside the existing key prefixes (`redis.go:27-32`), consistent with `vpn:status:<profile>` / `route:<target>` / `vpn:events`:

```
const metricsKeyPrefix = "metrics:"

// ACLDecisionCounterKey / TunnelDownCounterKey are the shared
        counter keys the
// acl-helper INCRs and the control-API reads at scrape (single
        source of truth).
func ACLDecisionCounterKey(decision string) string { return
        metricsKeyPrefix + "acl_decisions:" + decision } //
        decision = "OK"|"ERR"
func TunnelDownCounterKey() string                 { return
        metricsKeyPrefix + "tunnel_down_responses" }
```

Extend the `StatusBus` interface (`redis.go:37-52`):

```
// IncrCounter atomically adds `by` to a counter key, returning
        the new value.
IncrCounter(ctx context.Context, key string, by int64) (int64,
        error)
// GetCounter reads a counter key; a MISSING key is 0 (never
        fabricated), a
// transport error is surfaced so the collector can skip the
        series (§11.4.6).
GetCounter(ctx context.Context, key string) (int64, error)
```

5.2 `internal/redis/client.go` — implement on `*Client`

```
func (c *Client) IncrCounter(ctx context.Context, key string, by
        int64) (int64, error) {
    n, err := c.rdb.IncrBy(ctx, key, by).Result()
    if err != nil { return 0, fmt.Errorf("redis: incr %s: %w",
        key, err) }
    return n, nil
}

func (c *Client) GetCounter(ctx context.Context, key string)
        (int64, error) {
    n, err := c.rdb.Get(ctx, key).Int64()
    if errors.Is(err, goredis.Nil) { return 0, nil } // missing ⇒
        0 (never fabricated)
    if err != nil { return 0, fmt.Errorf("redis: get counter %s:
        %w", key, err) }
    return n, nil
}
```

5.3 `internal/aclhelper` — shared decision-recording policy (one place)

New `internal/aclhelper/record.go` so the ERR⇒tunnel_down mapping (§3) lives in ONE spot both the helper and the tests import. Extend `aclhelper.Bus` (`decide.go:14-21`) with `IncrCounter(ctx, key, by)` (the real `*redis.Client` satisfies it once 5.2 lands):

```
// RecordDecision increments the shared Redis counters for one
        verdict. Best-effort:
// the caller logs-and-continues on error — a metrics write NEVER
        alters or delays
// the fail-closed verdict (§11.4.6). ok=false increments BOTH the
        ERR series AND
// the tunnel_down counter (§3: every ERR ⇒ one ERR_TUNNEL_DOWN
        503).
func RecordDecision(ctx context.Context, bus Bus, ok bool) error {
    if ok {
        _, err := bus.IncrCounter(ctx,
        redis.ACLDecisionCounterKey("OK"), 1)
```

```
        return err
    }
    if _, err := bus.IncrCounter(ctx,
        redis.ACLDecisionCounterKey("ERR"), 1); err != nil {
        return err
    }
    _, err := bus.IncrCounter(ctx, redis.TunnelDownCounterKey(),
        1)
    return err
}
```

5.4 `cmd/acl-helper/main.go` `loop` — call it AFTER the reply is flushed

In `loop` (`main.go:84-110`), after the successful `w.Flush()` (line 100-102), record the decision on a bounded context. The reply is written FIRST, so a slow/failed `INCR` never delays Squid's answer nor changes the verdict:

```
// ... after ferr := w.Flush() succeeds, `ok` is already returned
        by decideWithTimeout:
recCtx, rcancel := context.WithTimeout(ctx, reqTimeout)
if rerr := aclhelper.RecordDecision(recCtx, dec.Bus, ok); rerr !=
        nil {
    fmt.Fprintln(os.Stderr, "acl-helper: metrics incr:", rerr) //
        best-effort, NEVER fatal
}
rcancel()
```

(`decideWithTimeout` at `main.go:96` already yields `ok`; thread it into the write branch.)

5.5 `internal/api/metrics.go` — Redis-reading const-metric collector (mirror `vpnUpCollector`)

Replace the in-process `aclDecisions *CounterVec` + `tunnelDownResponses Counter` (`metrics.go:43-47, 56-68`) with a collector that reads the shared keys at scrape:

```
type aclCounterCollector struct{ bus redis.StatusBus }

var (
    aclDecisionsDesc = prometheus.NewDesc(MetricACLDecisionsTotal,
        "Total external-acl decisions by outcome (decision=OK|
        ERR), from Redis.",
        []string{"decision"}, nil)
    tunnelDownDesc = prometheus.NewDesc(MetricTunnelDownResponses,

        "Total ERR_TUNNEL_DOWN 503 responses served (fail-closed),
        from Redis.", nil, nil)
)
```

```
func (c *aclCounterCollector) Describe(ch chan<-
        *prometheus.Desc) { ch <- aclDecisionsDesc; ch <-
        tunnelDownDesc }

func (c *aclCounterCollector) Collect(ch chan<-
        prometheus.Metric) {
    ctx, cancel := context.WithTimeout(context.Background(),
        2*time.Second)
    defer cancel()
    // Both decision series ALWAYS emit (missing key ⇒ 0),
        preserving the current
    // pre-touch-at-0 behaviour (metrics.go:66-68). Only a hard
        transport error
    // skips a series — never a fabricated value (§11.4.6, mirrors
        metrics.go:112).
    for _, d := range []string{"OK", "ERR"} {
        if v, err := c.bus.GetCounter(ctx,
        redis.ACLDecisionCounterKey(d)); err == nil {
            ch <- prometheus.MustNewConstMetric(aclDecisionsDesc,
        prometheus.CounterValue, float64(v), d)
        }
    }
    if v, err := c.bus.GetCounter(ctx,
        redis.TunnelDownCounterKey()); err == nil {
        ch <- prometheus.MustNewConstMetric(tunnelDownDesc,
        prometheus.CounterValue, float64(v))
    }
}
```

Register it in `newMetrics` beside the vpn_up collector (`metrics.go:69`):

```
reg.MustRegister(&aclCounterCollector{bus: bus})
```

`Metrics` now holds `bus redis.StatusBus`. The public `IncACLDecision` / `IncTunnelDownResponse` seam (`metrics.go:74-82`) is retained as thin convenience wrappers that write the SAME keys via `bus.IncrCounter` (so the existing in-process test path `metrics_test.go:32-33` still drives the real loop, and both processes converge on identical keys):

```
func (m *Metrics) IncACLDecision(decision string) {
    if decision != "OK" && decision != "ERR" { decision = "ERR" }
    _, _ = m.bus.IncrCounter(context.Background(),
        redis.ACLDecisionCounterKey(decision), 1)
    if decision == "ERR" { _, _ =
        m.bus.IncrCounter(context.Background(),
        redis.TunnelDownCounterKey(), 1) }
}
func (m *Metrics) IncTunnelDownResponse() { _, _ =
        m.bus.IncrCounter(context.Background(),
        redis.TunnelDownCounterKey(), 1) }
```

5.6 Interface ripple — fakes MUST gain the two methods

`aclhelper.Bus` and `redis.StatusBus` both grow `IncrCounter`/`GetCounter`, so every fake compiling against them MUST implement them or the packages will not build:

- `internal/api/harness_test.go:557-585 fakeBus` — add in-memory `IncrCounter`/`GetCounter` over a `map[string]int64` (guarded), plus a helper for baseline reads.
- any `aclhelper` decide-test fake Bus.

This is a compile-time forcing function (a fake that forgets the methods fails `go build`), not optional.

6. Redis key scheme (summary)

Key	Written by	Read by	Meaning
`metrics:acl_decisions:OK`	acl-helper `RecordDecision` (`INCR`)	API `aclCounterCollector` (`GET`)	`helix_proxy_a`
`metrics:acl_decisions:ERR`	acl-helper `RecordDecision` (`INCR`)	API `aclCounterCollector` (`GET`)	`helix_proxy_a`
`metrics:tunnel_down_responses`	acl-helper `RecordDecision` on ERR (`INCR`)	API `aclCounterCollector` (`GET`)	`helix_proxy_t`

Prefix `metrics:` is consistent with the existing `vpn:status:` / `route:` / `vpn:events` colon-delimited naming (`redis.go:27-32`). Keys are plain integer counters; no TTL (a counter must not expire mid-flight, unlike `vpn:status:` whose TTL IS the fail-closed mechanism).

7. Anti-bluff proving tests (§11.4.115 RED-polarity + §1.1 mutation)

7.1 In-process Go test — gate-able WITHOUT the live stack (PRIMARY guard)

New `internal/api/metrics_bytepath_test.go` using `fakeBus` (or real Redis when `REDIS_ADDR` is set, honest SKIP otherwise):

1. Scrape the registry → assert `acl_decisions_total{decision="OK"} == 0`, `tunnel_down_responses_total == 0` (baseline).

2. Simulate the acl-helper's write via the SHARED path: `aclhelper.RecordDecision(ctx, bus, true)` then `aclhelper.RecordDecision(ctx, bus, false)`.
3. Re-scrape → assert `{OK}` == 1, `{ERR}` == 1, `tunnel_down_responses_total` == 1.

This exercises write→Redis→collector→exposition in one process. **§11.4.115 polarity / §1.1 paired mutation:** point `aclCounterCollector` at a wrong key (e.g. `acl_decisions:OKX`) OR strip the `RecordDecision` ERR⇒tunnel_down line — the driven increment no longer appears in the scrape → the test FAILs. A test that still passes under either mutation is a bluff gate.

7.2 acl-helper-side integration test — real binary + real Redis

Extend `cmd/acl-helper/integration_test.go` (`TestIntegration_RealHelperRealRedis`, seeds route+status via the real client): after driving a request line that resolves to an UP route, assert `bus.GetCounter(redis.ACLDecisionCounterKey("OK"))` incremented; after a down/deleted-route line, assert `ERR` + `tunnel_down_responses` incremented. Honest §11.4.3 SKIP when Redis is unreachable (same pattern as the existing test, `integration_test.go:38-40`).

7.3 LIVE end-to-end — the §56 shell guard with the flag flipped (conductor-owned)

`tests/observability/metrics_scrape_test.sh` is already written for this: run it with `HELIX_METRICS_BYTEPATH_WIRED=1`. It drives a real `curl -x` through Squid → the acl-helper decides and `INCR`s Redis → re-scrape of `proxy-api /metrics` reads the incremented Redis counter → `DELTA>0`. A flat counter is then a HARD FAIL (`metrics_scrape_test.sh:250-254`). This is the §11.4.119 single-owner live proof; it needs the topology in §8.

8. Deployment topology requirement for the LIVE proof (load-bearing)

The live counter-increment proof needs the acl-helper (writer) and `proxy-api` (reader) pointed at the **same** Redis:

- `proxy-api` reads `REDIS_ADDR=proxy-redis:6379` (`docker-compose.observability.yml:109`).
- The dynamic Squid+acl-helper stack (`docker-compose.dynamic.yml`) must dial that same `proxy-redis`.

`docker-compose.observability.yml` today ADDs only `proxy-api`; the live proof therefore requires the dynamic proxy stack up **on the same Redis** as `proxy-api`, and the shell test's `HELIX_PROXY_URL` pointed at that Squid. Flag this as the enablement prerequisite for `HELIX_METRICS_BYTEPATH_WIRED=1`.

9. UNCONFIRMED / honest boundaries (§11.4.6)

- **UNCONFIRMED:** the committed spec (`docs/superpowers/specs/2026-06-30-vpn-aware-proxy-extension-design.md`) does NOT prescribe the increment MECHANISM for these two counters — §11 lists them as control-API metrics and §13 pairs egress with a generic "counter", but neither states Redis-counter vs pub/sub vs HTTP. The A1 recommendation is **derived by mirroring the CONFIRMED `helix_proxy_vpn_up`-from-Redis pattern** (`metrics.go:89-123`), not read from an explicit P5/P10 sentence. `config/grafana/README.md:29` calls the tunnel-down metric a *"PLANNED custom control-plane counter (acl-helper → deny_info 503), built P5/P6 … confirmed/renamed P10"* — consistent with A1's acl-helper-writer direction but silent on the transport.
- **CONFIRMED (not a guess):** the ERR⇒503 1:1 mapping (§3) is from `routing.go:295-297`; the acl-helper's Redis-only dependency from `main.go:13`; the API's scrape-time Redis read pattern from `metrics.go:89-123`; the API's Redis connectivity from `docker-compose.observability.yml:109`.
- **Counter-reset semantics:** Redis-backed counters do NOT reset on API restart (better than in-process); a Redis flush resets them (normal, Prometheus `rate()` tolerates it).

10. Recommendation

Adopt **Option A1**. It is the single-architecture mirror of the already-shipped `vpn_up`-from-Redis collector, keeps the acl-helper's Redis-only surface, is lossless and restart-safe, and makes the already-written §56 guard's `HELIX_METRICS_BYTEPATH_WIRED=1` path go green on real proxied traffic. Files to change: `internal/redis/redis.go`, `internal/redis/client.go`, `internal/aclhelper/record.go` (new) + `decide.go` (Bus interface), `cmd/acl-helper/main.go` (`loop`), `internal/api/metrics.go` — plus the fake-bus ripple in `internal/api/harness_test.go` and the three proving tests in §7.

Sources verified

- `control-plane/internal/api/metrics.go` (lines 14-17, 43-82, 89-123) — accessed 2026-07-01.

- `control-plane/cmd/acl-helper/main.go` (lines 1-13, 62-122) — accessed 2026-07-01.
- `control-plane/internal/aclhelper/decide.go` (lines 14-70) — accessed 2026-07-01.
- `control-plane/internal/redis/redis.go` (lines 27-52) + `client.go` (lines 42-164) — accessed 2026-07-01.
- `control-plane/internal/routing/routing.go` (lines 281-297) — accessed 2026-07-01.
- `tests/observability/metrics_scrape_test.sh` (lines 225-265) — accessed 2026-07-01.
- `docker-compose.observability.yml` (line 109) + `docker-compose.dynamic.yml` (line 223) — accessed 2026-07-01.